CH3: Common Mistakes in Declarations and Definitions

1. Every pointer must have a defined type

In C language, declaring a pointer without specifying its type is incomplete.
The compiler must know what kind of variable the pointer will refer to.

For example:

```
WORD *p; // pointer to WORD type
WORD *q; // another pointer to WORD type
```

Wrong declaration:

```
*p;// incomplete, type information missing
*q;// incomplete, type information missing
```

- The asterisk (*) alone is not enough. It must be combined with a data type to indicate what type of value the pointer will reference.

2. Assigning addresses without type casting

If you assign a constant address directly to a pointer, a **type cast** is required.

```
WORD *p;
p = (WORD *)0x1000; // correct: explicit type conversion
```

Wrong declaration:

```
WORD *p;
p = 0x1000; // compiler error: assigning integer to pointer without cast
```

- The compiler treats 0x1000 as an integer, not as an address.
  Casting to (WORD *) tells the compiler that p now points to the address 0x1000.

3. Confusing address and value operations

```
WORD *p;
WORD x = 42;

p = &x;    // store address of x
*p = 10;   // store 10 at the memory location pointed by p
```

Wrong declaration:

```
p = x;        // assigns the value of x, not its address
```

- The & operator gives the address of a variable.
  The * operator accesses the value stored at that address.
  Forgetting or misplacing these operators is one of the most common pointer mistakes.

4. Dereferencing invalid or uninitialized pointers

```
WORD *p;
*p = 42; // undefined behavior! p is not pointing to a valid memory location
```

To fix it:

```
WORD *p;
WORD x;
p = &x;  // assign valid address first
*p = 42; // now valid: stores 42 in x
```

- Always make sure a pointer holds a valid address before dereferencing it. Dereferencing uninitialized pointers can crash the program or corrupt memory.

Operation	Meaning	Example
p = &x;	p holds the address of x	pointer assignment
*p = x;	value of x is copied into memory pointed by p	dereference
p = (WORD *)0x1000;	pointer now points to address 0x1000	direct addressing

5. Misusing typedef with pointers

typedef helps make code more readable, but it can also hide pointer types if used carelessly.

```
typedef WORD *WORDPTR;
WORDPTR x, y;       //  both x and y are pointers to WORD
```

Without typedef, this would happen:

```
WORD *x, y;       // only x is a pointer; y is a normal WORD variable
```

- Use typedef to simplify syntax, but clearly document that the new type represents a pointer.

Prefer the typedef below:

```
int (*operation)(int, int);  // normal definition
```

by the help of typedef:

```
typedef int (*OPERATION)(int, int);  // new definition

OPERATION add, sub;   // it can be easily called
```

6. Misunderstanding void* Usage

```
int x = 42;
void *pv = &x;
printf("%d\n", *(int *)pv);     // correct: cast before use
```

but:

```
printf("%d\n", *pv);            // compiler error: unknown type
```

7. Returning Pointers to Local Variables

```
int* wrongFunc() {
    int x = 10;
    return &x;      //  invalid: x no longer exists after return
}
```

- The problem here is this: x is created in stack memory (automatic memory). When the function ends, that memory area is automatically deleted.

Correct:

```
int* correctFunc() {
    int *p = malloc(sizeof(int));
    *p = 10;
    return p;       // valid: memory stays allocated after function ends
}
```

- The memory allocated with malloc() is kept in the heap, and the data there is not deleted even if the function ends.

(The book does not go into this part in detail, but I would like to give additional detailed information in section 8.)

**8. What should I do if I don't know how much memory to allocate?

8.1 If you learn the size later — use realloc()

You can allocate a small block first and then expand it when needed.

```
int *arr = malloc(10 * sizeof(int));   // allocate for 10 elements

// later, if you need more space:
arr = realloc(arr, 20 * sizeof(int));  // resize to 20 elements
```

8.2 If the size is determined by the user

When the number of elements is not known until runtime, ask the user.

```
int n;
printf("How many elements do you need? ");
scanf("%d", &n);
int *arr = malloc(n * sizeof(int));   // dynamic allocation based on input
```

8.3 If you can determine the size beforehand

For example, you're reading data from a file but don't know how many lines it has.
You can count first, then allocate the exact amount.

```
int count = 0;
while (fgets(line, sizeof(line), file))
    count++;

rewind(file);
int *data = malloc(count * sizeof(int));   // allocate based on count
```

8.4 If the data keeps growing dynamically

When data size increases gradually and unpredictably, use incremental reallocation to expand memory step by step.

```
int *arr = NULL;
int size = 0, used = 0;

while (has_more_data()) {
    if (used == size) {
        size = (size == 0) ? 10 : size * 2;        // double the capacity
        arr = realloc(arr, size * sizeof(int));    // grow the array
    }
    arr[used++] = get_new_data();
}
```

NOTE: In C++, std::vector provides automatic dynamic resizing, but in C, a similar feature can be implemented by creating a small custom vector library; otherwise, memory management must be handled manually using malloc, realloc, and free.

8.5 If you have no exact size but a rough estimate

Start with a reasonable guess and expand later if needed:

```
#define INITIAL_SIZE 100
int *data = malloc(INITIAL_SIZE * sizeof(int));
```

Situation	Strategy	Function(s) Used
Size comes from user	Read input → allocate	scanf, malloc
Size increases dynamically	Resize memory	realloc
Size can be counted first	Count → then allocate	malloc
Size unknown, rough guess	Start small → grow later	malloc + realloc

NOTE1: Don't forget to free(arr); at the end

```
free(arr);                          // release memory
arr = NULL;                         // avoid dangling pointer
```

NOTE2: When should you free()?

Free() a memory location as soon as you're finished using it. But If you're still going to use that data, free()ing it early is a bad idea. Because free() returns the memory, the data at that address is no longer guaranteed.

NOTE3: Without free(), the program keeps 4 MB(1,000,000 integers) of memory every time it runs.

```
int *p = malloc(1000000 * sizeof(int));   // 1 million int = 4 MB for an example
```

A basic example for free()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(3 * sizeof(int));  // 3 adet int için yer aç
    p[0] = 10;
    p[1] = 20;
    p[2] = 30;

    printf("Before free: %d %d %d\n", p[0], p[1], p[2]);

    free(p);   // release the memory
    p = NULL;  // now unvalid, became null

    // The location pointed to by p is now deleted and cannot be used!
    // printf("%d", p[0]);  If you do this, the program will crash!

    return 0;
}
```

9. Misunderstanding pointer arithmetic

```
WORD x, *p = (WORD *)0x1000;
x = *(p + 0);   // reads from address 0x1000
x = *(p + 1);   // reads from address 0x1002  (WORD = 2 bytes)
x = *(p + 2);   // reads from address 0x1004
```

Adding 1 to a pointer increases the address by the **size of its data type**, not by 1 byte.

- char *p → p + 1 moves **1 byte forward**
- int *p → p + 1 moves **4 bytes forward**
- double *p → p + 1 moves **8 bytes forward**

10.1 Confusing *p++ and (*p)++

```
*p++       // use the value p points to, then move p to next element
(*p)++     // increase the value stored at p, but p itself doesn't move
```

10.2 Dangerous usage: *(p++)++

Dangerous usage: *(p++)++

- Execution order becomes compiler-dependent.
    1. p++ → uses old pointer, then moves to next.
    2. * → accesses the value at old pointer.
    3. ++ → increments that value.

Result is not portable! Different compilers may interpret it differently.